EXPERIMENT 2: TITANIC DATASET ANALYSIS

AIM:

To analyze the Titanic dataset using Python by inspecting dataset information and summary statistics, handling missing values, visualizing passenger-class distribution, filtering passengers based on specified conditions, and splitting the data into training and testing sets.

ALGORITHM:

1. Import the required libraries: Pandas, Seaborn, Matplotlib, SimpleImputer, and train_test_split.
2. Load the Titanic CSV dataset into a Pandas DataFrame and display the first five rows.
3. Display the dataset shape, information, and summary statistics.
4. Fill missing values in the Age column using SimpleImputer with the mean strategy.
5. Replace missing values in the Cabin column with “Unknown”.
6. Fill missing values in the Embarked column using its mode.
7. Plot the passenger class distribution using a count plot.
8. Display female passengers who survived.
9. Display third-class passengers younger than 18 years.
10. Display first-class passengers older than 40 years.
11. Display surviving passengers in the specified category.
12. Display male passengers whose fare is greater than 100.
13. Display passengers who embarked from C and belonged to passenger class 2.
14. Display passengers with SibSp greater than 2.
15. Display non-surviving passengers who had no siblings/spouses and no parents/children aboard.
16. Display the five oldest surviving passengers.
17. Display passengers who paid zero fare.
18. Split the dataset into training and testing sets with test_size=0.2 and random_state=42.

Step 1: Load the Titanic Dataset and Convert it into a DataFrame

```
#EXP_NO_2
#Sanjay_S_241801246
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

from google.colab import drive
drive.mount('/content/drive')

file_path = "/content/drive/MyDrive/PODS/Titanic-Dataset.csv"
df = pd.read_csv(file_path)
```

```
Mounted at /content/drive
```

Step 1: Load the Titanic Dataset and Convert it into a DataFrame

```
#EXP_NO_2
#Sanjay_S_241801246
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

from google.colab import drive
drive.mount('/content/drive')

file_path = "/content/drive/MyDrive/PODS/Titanic-Dataset.csv"
df = pd.read_csv(file_path)
```

```
Mounted at /content/drive
```

Step 2: Understand the Data (Shape, Info, Describe)

```
#EXP_NO_2
#Sanjay_S_241801246

df.shape
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Step 3: Fill the Missing Values in Age using SimpleImputer

```
#EXP_NO_2
#Sanjay_S_241801246
imputer = SimpleImputer(strategy='mean')
df["Age"] = imputer.fit_transform(df[["Age"]])
print(df["Age"].isnull().sum())
```

```
0
```

Step 4: Fill Missing Cabin with "Unknown"

```
#EXP_NO_2
#Sanjay_S_241801246
df["Cabin"] = df["Cabin"].fillna("Unknown")
print(df["Cabin"].isnull().sum())
```

```
0
```

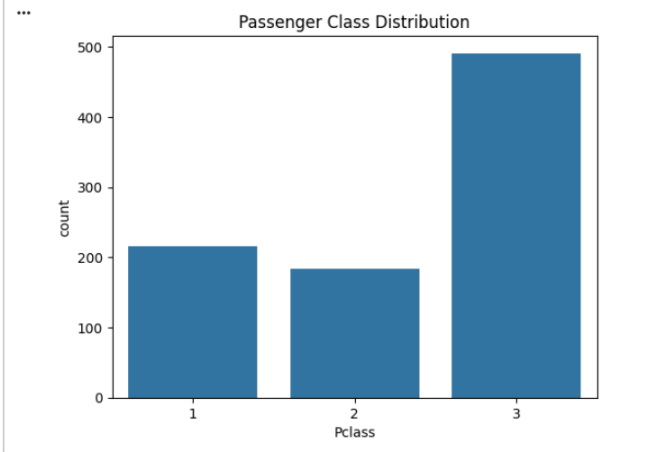
Step 5: Fill Missing Embarked with the Mode

```
#EXP_NO_2
#Sanjay_S_241801246
mode_value = df["Embarked"].mode()[0]
df["Embarked"] = df["Embarked"].fillna(mode_value)
print(df["Embarked"].isnull().sum())
```

```
0
```

Step 6: Analyse the Passenger Class Feature using Countplot

```
#EXP_NO_2
#Sanjay_S_241801246
sns.countplot(x="Pclass", data=df)
plt.title("Passenger Class Distribution")
plt.show()
```



Step 7: List the Names of Passengers who are Female and Survived

```
#EXP_NO_2
#Sanjay_S_241801246
female_survived = df[(df["Sex"] == "female") & (df["Survived"] == 1)]
female_survived[["Name"]]
```

	Name
1	Cumings, Mrs. John Bradley (Florence Briggs Th...
2	Heikkinen, Miss. Laina
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)
8	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9	Nasser, Mrs. Nicholas (Adele Achem)
...	...
874	Abelson, Mrs. Samuel (Hannah Wizosky)
875	Najib, Miss. Adele Kiamie "Jane"
879	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
880	Shelley, Mrs. William (Imanita Parrish Hall)
887	Graham, Miss. Margaret Edith

233 rows × 1 columns

Step 8: Find all Passengers who are in 3rd Class and Under the Age of 18

```
#EXP_NO_2
#Sanjay_S_241801246
third_class = df[(df["Pclass"] == 3) & (df["Age"] < 18)]
third_class[["Name", "Age", "Pclass"]]
```

	Name	Age	Pclass
7	Palsson, Master. Gosta Leonard	2.0	3
10	Sandstrom, Miss. Marguerite Rut	4.0	3
14	Vestrom, Miss. Hulda Amanda Adolfina	14.0	3
16	Rice, Master. Eugene	2.0	3
22	McGowan, Miss. Anna "Annie"	15.0	3
...	...	...	...
844	Culumovic, Mr. Jeso	17.0	3
850	Andersson, Master. Sigvard Harald Elias	4.0	3
852	Boulos, Miss. Nourelain	9.0	3
869	Johnson, Master. Harold Theodor	4.0	3
875	Najib, Miss. Adele Kiamie "Jane"	15.0	3

78 rows × 3 columns

Step 9: Find the List of Passengers who are in Class 1 and Older than 40

```
#EXP_NO_2
#Sanjay_S_241801246
class1 = df[(df["Pclass"] == 1) & (df["Age"] > 40)]
class1[["Name", "Age", "Pclass"]]
```

	Name	Age	Pclass
6	McCarthy, Mr. Timothy J	54.0	1
11	Bonnell, Miss. Elizabeth	58.0	1
35	Holverson, Mr. Alexander Oskar	42.0	1
52	Harper, Mrs. Henry Sleeper (Myna Haxtun)	49.0	1
54	Ostby, Mr. Engelhart Cornelius	65.0	1
...	...	...	...
856	Wick, Mrs. George Dennick (Mary Hitchcock)	45.0	1
857	Daly, Mr. Peter Denis	51.0	1
862	Swift, Mrs. Frederick Joel (Margaret Welles Ba...	48.0	1
871	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	47.0	1
879	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	56.0	1

76 rows × 3 columns

Step 10: Find the List of Passengers Survived in the Above Category

```
#EXP_NO_2
#Sanjay_S_241801246
survived_class1 = df[(df["Pclass"] == 1) &
                     (df["Age"] > 40) &
                     (df["Survived"] == 1)]
survived_class1[["Name", "Age", "Pclass", "Survived"]]
```

299	Baxter, Mrs. James (Helene DeLaudeniere Chaput)	50.0	1	1
337	Burns, Miss. Elizabeth Margaret	41.0	1	1
366	Warren, Mrs. Frank Manley (Anna Sophia Atkinson)	60.0	1	1
380	Bidois, Miss. Rosalie	42.0	1	1
449	Peuchen, Major. Arthur Godfrey	52.0	1	1
453	Goldenberg, Mr. Samuel L	49.0	1	1
460	Anderson, Mr. Harry	48.0	1	1
496	Eustis, Miss. Elizabeth Mussey	54.0	1	1
513	Rothschild, Mrs. Martin (Elizabeth L. Barrett)	54.0	1	1
523	Hippach, Mrs. Louis Albert (Ida Sophia Fischer)	44.0	1	1
556	Duff Gordon, Lady. (Lucille Christiana Sutherl...	48.0	1	1
571	Appleton, Mrs. Edward Dale (Charlotte Lamson)	53.0	1	1
587	Frolicher-Stehli, Mr. Maxmillian	60.0	1	1
591	Stephenson, Mrs. Walter Bertram (Martha Eustis)	52.0	1	1
599	Duff Gordon, Sir. Cosmo Edmund ("Mr Morgan")	49.0	1	1
621	Kimball, Mr. Edwin Nelson Jr	42.0	1	1

Step 11: Which Male Passengers Paid a Fare Greater than 100?

```
#EXP_NO_2
#Sanjay_S_241801246
male_fare = df[(df["Sex"] == "male") & (df["Fare"] > 100)]
male_fare[["Name", "Fare"]]
```

	Name	Fare
27	Fortune, Mr. Charles Alexander	263.0000
118	Baxter, Mr. Quigg Edmond	247.5208
305	Allison, Master. Hudson Trevor	151.5500
332	Graham, Mr. George Edward	153.4625
373	Ringhini, Mr. Sante	135.6333
377	Widener, Mr. Harry Elkins	211.5000
390	Carter, Mr. William Ernest	120.0000
438	Fortune, Mr. Mark	263.0000
505	Penasco y Castellana, Mr. Victor de Satode	108.9000
527	Farthing, Mr. John	221.7792
544	Douglas, Mr. Walter Donald	106.4250

Step 12: List Passengers who Embarked from Port 'C' and were in Class 2

```
#EXP_NO_2
#Sanjay_S_241801246
embarked_c = df[(df["Embarked"] == "C") & (df["Pclass"] == 2)]
embarked_c[["Name", "Embarked", "Pclass"]]
```

	Name	Embarked	Pclass
9	Nasser, Mrs. Nicholas (Adele Achem)	C	2
43	Laroche, Miss. Simonne Marie Anne Andree	C	2
122	Nasser, Mr. Nicholas	C	2
135	Richard, Mr. Emile	C	2
181	Pernot, Mr. Rene	C	2
292	Levy, Mr. Rene Jacques	C	2
308	Abelson, Mr. Samuel	C	2
361	del Carlo, Mr. Sebastiano	C	2
389	Lehmann, Miss. Bertha	C	2
473	Jerwan, Mrs. Amin S (Marie Marthe Thuillard)	C	2
547	Padro y Manent, Mr. Julian	C	2
608	Laroche, Mrs. Joseph (Juliette Marie Louise La...	C	2
685	Laroche, Mr. Joseph Philippe Lemercier	C	2
817	Mallet, Mr. Albert	C	2

Step 13: Show all Passengers who had More than 2 Siblings/Spouses Aboard (SibSp > 2)

```
#EXP_NO_2
#Sanjay_S_241801246
sibsp = df[df["SibSp"] > 2]
sibsp[["Name", "SibSp"]]
```

171	Rice, Master. Arthur	4
176	Lefebre, Master. Henry Forbes	3
180	Sage, Miss. Constance Gladys	8
182	Asplund, Master. Clarence Gustaf Hugo	4
201	Sage, Mr. Frederick	8
229	Lefebre, Miss. Mathilde	3
233	Asplund, Miss. Lillian Gertrud	4
261	Asplund, Master. Edvin Rojj Felix	4
266	Panula, Mr. Ernesti Arvid	4
278	Rice, Master. Eric	4
324	Sage, Mr. George John Jr	8
341	Fortune, Miss. Alice Elizabeth	3
374	Palsson, Miss. Stina Viola	3
386	Goodwin, Master. Sidney Leonard	5

Step 14: List the Passengers who did not Survive and had no Family Aboard

```
#EXP_NO_2
#Sanjay_S_241801246
no_family = df[(df["Survived"] == 0) &
               (df["SibSp"] == 0) &
               (df["Parch"] == 0)]
no_family[["Name", "Survived", "SibSp", "Parch"]]
```

	Name	Survived	SibSp	Parch
4	Allen, Mr. William Henry	0	0	0
5	Moran, Mr. James	0	0	0
6	McCarthy, Mr. Timothy J	0	0	0
12	Saundercock, Mr. William Henry	0	0	0
14	Vestrom, Miss. Hulda Amanda Adolfina	0	0	0
...	...	...	...	...
882	Dahlberg, Miss. Gerda Ulrika	0	0	0
883	Banfield, Mr. Frederick James	0	0	0
884	Sutehall, Mr. Henry Jr	0	0	0
886	Montvila, Rev. Juozas	0	0	0
890	Dooley, Mr. Patrick	0	0	0

374 rows × 4 columns

Step 15: Find the Top 5 Oldest Passengers who Survived

```
#EXP_NO_2
#Sanjay_S_241801246
top5 = df[df["Survived"] == 1].sort_values(by="Age", ascending=False)
top5[["Name", "Age"]].head(5)
```

	Name	Age
630	Barkworth, Mr. Algernon Henry Wilson	80.0
275	Andrews, Miss. Kornelia Theodosia	63.0
483	Turkula, Mrs. (Hedwig)	63.0
570	Harris, Mr. George	62.0
829	Stone, Mrs. George Nelson (Martha Evelyn)	62.0

Step 16: List Passengers who Paid Zero Fare

```
#EXP_NO_2
#Sanjay_S_241801246
zero_fare = df[df["Fare"] == 0]
zero_fare[["Name", "Fare"]]
```

	Name	Fare
179	Leonard, Mr. Lionel	0.0
263	Harrison, Mr. William	0.0
271	Tornquist, Mr. William Henry	0.0
277	Parkes, Mr. Francis "Frank"	0.0
302	Johnson, Mr. William Cahoone Jr	0.0
413	Cunningham, Mr. Alfred Fleming	0.0
466	Campbell, Mr. William	0.0
481	Frost, Mr. Anthony Wood "Archie"	0.0
597	Johnson, Mr. Alfred	0.0
633	Parr, Mr. William Henry Marsh	0.0

Step 17: Split the Dataset for Training and Testing

```
#EXP_NO_2
#Sanjay_S_241801246
X = df.drop("Survived", axis=1)
y = df["Survived"]

X = pd.get_dummies(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (712, 1731)
Testing Data Shape: (179, 1731)
```

The Titanic dataset was successfully loaded and analyzed. Dataset information and summary statistics were displayed, missing values in Age, Cabin, and Embarked were handled, and passenger-class distribution was visualized. Passenger records were filtered using the specified conditions. Finally, the dataset was split into 712 training records and 179 testing records.